

Session 1 Homework — Add a "page not found" page

Context

This extends the same B4F Hub project from class. It is **not** a separate mini-project, and it is not the Thursday weekly mini-project: this is Tuesday's continuity homework.

Right now, if someone opens an address that matches none of our pages — for example

`http://localhost:5173/this-does-not-exist` — the app shows an empty page. Nothing tells the visitor what happened.

Goal

Give visitors something useful whenever they open an address that does not exist.

What to build

1. A new "page not found" page with a clear message that the address they tried does not exist.
2. A link on that page that takes the visitor back to Home (`/`).
3. Make it show for **any** address that does not match one of the app's real pages: Home, Community, Opportunities, and an opportunity's detail page.

Rules

Allowed — everything from Session 1: creating a page component in its own file, registering a route with a path and an element, and `Link`.

New syntax allowed for this task only: a route's `path` can be the special value `"*"`, which means "anything that no other route matched". You do not need to learn any other new routing idea: it is just a different string in the same `path` you have already used several times.

Not yet — nested routes, shared layouts, redirects, protected pages, and reading values from the address in a new way. None is needed here.

Test it

- Visit a made-up address such as `/this-does-not-exist`: your page shows.
- Visit `/`, `/community`, `/opportunities`, and a real opportunity's detail address (for example `/opportunities/101`): all of them still work exactly as before. Your new route must not "catch" an address that already has a real page.
- Click the link on your page: it takes you to `/`.

Done when

- ☐ An unknown address shows your page instead of an empty screen.
- ☐ Every existing page still works.
- ☐ Your page has a working link back to Home.
- ☐ `npm run build` and `npm run lint` (inside `client/`) both pass.
- ☐ No console errors in the browser.

Working time

About 25–40 minutes for one new page file, one import, one route and a little CSS.

Hint

Write your catch-all route last among your routes: it then reads naturally as "anything else, catch it here". (This version of the router actually picks the most specific matching route rather than the first one, so it would not break if you put it elsewhere, but a person reading the file top to bottom expects the specific routes first and the fallback last.)